

CityTech TTP Summer Bootcamp

Joins, Database Systems, and ORMs

2026-06-23

Agenda

1. Review
2. SQL Joins
3. Data: Clean vs Dirty, Structured vs Unstructured
4. Quotes and SQL Injection
5. Database Systems (MySQL, PostgreSQL, MongoDB)
6. DBMSs and pgAdmin
7. ORMs and Prisma
8. SQL Games

Review

What are the most important things you remember from yesterday?

Follow Up: SQL Joins

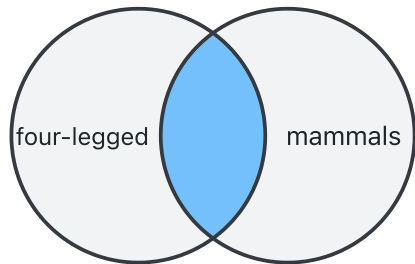
A **join** combines rows from two tables. The join **type** decides which rows you keep.

Picture the two tables as overlapping circles, matched on a join key (the animal's name):

- **Left table:** `four_legged` animals
- **Right table:** `mammals`
- The **overlap** = animals in both (four-legged mammals)

Inner Join (just **JOIN**)

Keeps only rows that **match in both** tables.

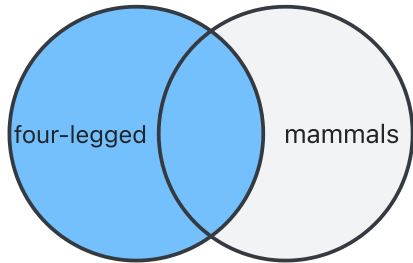


```
SELECT * FROM four_legged JOIN mammals ON four_legged.name = mammals.name;
```

Animals that are both four-legged and mammals (dog, cat, horse). Everything else is **dropped**.

Left Join

Keeps **every row from the left** table, plus matches from the right (**NULL** where none).

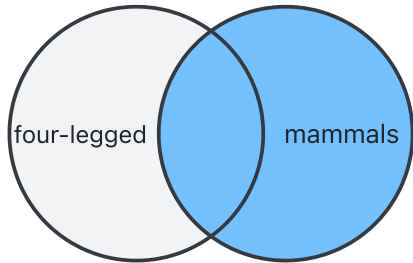


```
SELECT * FROM four_legged LEFT JOIN mammals ON four_legged.name = mammals.name;
```

Every four-legged animal. Non-mammals like lizards and crocodiles still appear, with **mammals** columns **NULL**.

Right Join

Keeps **every row from the right** table, plus matches from the left (**NULL** where none).

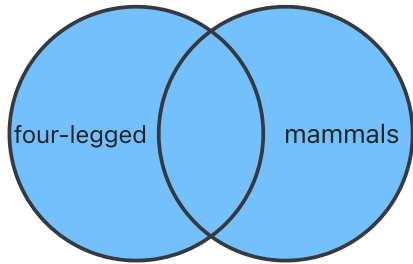


```
SELECT * FROM four_legged RIGHT JOIN mammals ON four_legged.name = mammals.name;
```

Every mammal. Non-four-legged ones like humans, whales, and bats still appear, with **four_legged** columns **NULL**.

Full Outer Join

Keeps **all rows from both** tables, matched where possible (`NULL` on the missing side).



```
SELECT * FROM four_legged FULL OUTER JOIN mammals ON four_legged.name = mammals.name;
```

Every animal from either list: matched where both apply, `NULL`-filled where only one does.

Follow Up: Dirty vs Clean Data

Real-world data is often **dirty**: messy, inconsistent, or incomplete. Cleaning it is a big part of working with data.

- **Dirty data**: missing values, duplicates, inconsistent formats, typos, or wrong types
- **Clean data**: consistent, complete, and correctly formatted, so you can query it reliably
- **Data cleaning** is the work of turning dirty into clean

Dirty vs Clean: An Example

Dirty

name	species	age
Rex	DOG	3
bella	dog	
Max	Dog	three

Clean

name	species	age
Rex	dog	3
Bella	dog	2
Max	dog	3

Dirty has inconsistent casing (DOG , Dog), a missing age, and three written as a word. Clean is one consistent, complete format.

Follow Up: Structured vs Unstructured Data

- **Structured data:** organized into a fixed shape, rows and columns with a schema. Easy to store in a database and query with SQL
- **Unstructured data:** no predefined model, so it doesn't fit neatly into tables. You can store it, but it's harder to query directly
- In between is **semi-structured** data like JSON: it has structure, but no fixed table schema

Structured vs Unstructured: An Example

Structured

name	species	age
Rex	dog	3
Bella	cat	2

Unstructured

"Rex is a friendly 3-year-old dog who loves long walks and napping in the sun."

Photos, emails, audio clips, PDFs...

Structured fits neatly into a table you can query with SQL. Unstructured (free text, images, audio) can hold the same facts but has no rows or columns to query.

Follow Up: Quotes and Backticks in SQL

Three quote characters, three different jobs:

- **Single quotes** `'...'`: a **string value** (text data)
- **Double quotes** `"..."`: an **identifier** (a table or column name) in standard SQL
- **Backticks** do the same as double quotes, but only in **MySQL / MariaDB**

```
WHERE species = 'dog'      -- 'dog' is a text value
SELECT "order" FROM t      -- "order" is a column name (PostgreSQL, SQLite)
SELECT `order` FROM t      -- `order` is a column name (MySQL, MariaDB)
```

A Common Gotcha

Always use **single quotes** for string values:

```
WHERE name = 'Rex'    -- correct: 'Rex' is a string  
WHERE name = "Rex"    -- risky: standard SQL reads "Rex" as a column name
```

- It might work in MySQL, but PostgreSQL and SQLite treat `"Rex"` as an **identifier**, not text
- Rule of thumb: `'...'` for values, `"..."` for names

Follow Up: MySQL

A widely used **relational** database, the classic web-app default.

- **Popular since:** the late 1990s, riding the **LAMP** stack (first released 1995)
- **Trend:** still #2 overall, but slowly losing developer mindshare to PostgreSQL
- **Strengths:** easy to start, everywhere, great for read-heavy web apps; powers WordPress
- **Weaknesses:** looser SQL-standard support and fewer advanced features; Oracle-owned (hence the **MariaDB** fork)

Follow Up: PostgreSQL

A powerful, standards-focused **relational** database.

- **Popular since:** surged through the late 2010s and 2020s (roots at UC Berkeley, 1986)
- **Trend:** the momentum leader, now the **most-used** database among developers
- **Strengths:** feature-rich (JSONB, arrays, window functions, extensions like PostGIS); strong data integrity and complex queries
- **Weaknesses:** a slightly steeper learning curve and heavier setup than MySQL

Follow Up: MongoDB

A **non-relational** (NoSQL) **document** database that stores JSON-like documents.

- **Popular since:** the early 2010s, with the rise of JavaScript and JSON apps (released 2009)
- **Trend:** still the leading document database, though many teams now reach for PostgreSQL's JSONB
- **Strengths:** flexible schema, fast to prototype, scales horizontally; a natural fit for JSON
- **Weaknesses:** weaker at joins and consistency; not built for complex relational queries

Follow Up: **INSERT INTO**

INSERT INTO adds new rows to a table:

```
INSERT INTO pets (name, species) VALUES ('Rex', 'dog');
```

- List the **columns**, then the matching **values**
- Add several rows at once with more comma-separated **(...)** groups
- The values often come from **user input**, which is where danger creeps in

Security: SQL Injection

Building SQL by **gluing in user input** lets a user inject their own SQL:

```
// DANGER: user input pasted straight into the query  
db.query("INSERT INTO students (name) VALUES ('" + name + "')");
```

If `name` is `Robert'); DROP TABLE students;--`, the database runs the `DROP TABLE` too.

"Little Bobby Tables"



A school trusted a name field straight from a form. XKCD 327, "Exploits of a Mom":

<https://xkcd.com/327/>

Parameterized Queries (the Fix)

Never concatenate user input. Use **placeholders** and pass the values separately, so the database treats them as **data, not code**:

```
// safe: the value is bound to the ? placeholder  
db.query("INSERT INTO students (name) VALUES (?)", [name]);
```

- The driver escapes the value for you, so injection can't happen
- Most libraries and **ORMs** do this by default (another reason to use them)
- Postgres uses `$1, $2` ; SQLite and MySQL use `?` placeholders

What Is a DBMS?

A **database management system (DBMS)** is the software that **stores, organizes, and manages** a database, giving apps a controlled way to access it.

It handles the hard parts for you:

- **Querying:** run SQL and return results efficiently (indexes)
- **Concurrency:** many users reading and writing at once, safely
- **Integrity:** enforce rules (keys, types, constraints)
- **Transactions:** all-or-nothing changes (`COMMIT` / `ROLLBACK`)

SQLite vs PostgreSQL

Both are **relational DBMSs** that speak SQL. The real difference is **architecture**:

	SQLite	PostgreSQL
Type	embedded (a file)	client-server
Setup	none	run a server
Concurrent writers	one at a time	many
Users / permissions	none	built in
Best for	learning, small or single-app	multi-user, production apps

The SQL you write is nearly identical in both.

pgAdmin

pgAdmin is a **graphical tool for managing PostgreSQL**. It's a client you point at a Postgres server, not a database itself.

- Browse **servers, databases, schemas, and tables** visually
- Write and run queries in a **query editor**, and view results in a grid
- Manage **users, permissions, and settings** by clicking
- View diagrams and monitor the server

Postgres also has a command-line client, `psql` ; pgAdmin is the visual alternative.

Code Along: Postgres with pgAdmin

Let's connect to PostgreSQL in pgAdmin and build a **cities and sports teams** database together.

pgAdmin: Docs and Tutorials

Yes. pgAdmin is well documented, and the docs include a built-in tutorial.

- **Official documentation:** <https://www.pgadmin.org/docs/>
- **Getting Started guide** (within the docs): connecting to a server, using the Query Tool, and managing database objects

For PostgreSQL itself, see the **official docs**: <https://www.postgresql.org/docs/>

Ports: Postgres, Express, and React

Each server is its own program listening on its own **port** on `localhost`:

Server	Default port
PostgreSQL	5432
Express (your Node API)	3000
React dev server (<code>yarn dev</code> , Vite)	5173

Because the ports differ, all three can run **at the same time**: your React app (`5173`) calls your Express API (`3000`), which queries Postgres (`5432`).

What Is an ORM?

An **ORM** (Object-Relational Mapping) lets you work with a database using your **language's objects** instead of writing raw SQL.

It maps:

- a **table** to a **class** (a "model")
- a **row** to an **object**
- a **column** to a **property**

Instead of `SELECT * FROM cities WHERE id = 1`, you write something like `City.findById(1)` and the ORM generates the SQL.

ORMs: Tools and Tradeoffs

- Because ORMs use **parameterized queries** under the hood, they guard against **SQL injection** by default (no more Bobby Tables)
- Node examples: **Prisma, Sequelize, Drizzle**
- Convenient and language-native, but they can **hide the SQL** that actually runs

Introducing Prisma

Prisma is a popular TypeScript ORM. You describe your data in a **schema**, and Prisma generates a fully typed client.

```
// schema.prisma
model City {
  id      String @id @default(uuid())
  name    String
  teams   Team[]
}

model Team {
  id      String @id @default(uuid())
  name    String
  sport   String
  city    City   @relation(fields: [cityId], references: [id])
  cityId  String
}
```

SQL vs Prisma: A Join

Find all baseball teams with their city, sorted by name.

```
SELECT teams.name, cities.name AS city
FROM teams
JOIN cities ON teams.cityId = cities.id
WHERE teams.sport = 'baseball'
ORDER BY teams.name;
```

```
const teams = await prisma.team.findMany({
  where: { sport: "baseball" },
  include: { city: true },
  orderBy: { name: "asc" },
});
```

SQL vs Prisma: Inserting

Add a new city. Its `id` is a **UUID generated automatically**, so you don't set it.

```
INSERT INTO cities (name) VALUES ('Boston');
```

```
const city = await prisma.city.create({  
  data: { name: "Boston" },  
});
```


Code Along: A Prisma Node.js Script

Let's write a small Node script that talks to a database through Prisma, together.

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

const city = await prisma.city.create({ data: { name: "New York" } });
await prisma.team.create({
  data: { name: "Yankees", sport: "baseball", cityId: city.id },
});

const teams = await prisma.team.findMany({ include: { city: true } });
console.log(teams);
```

Then we'll extend it: a filter, grouping, and an update.

Using AI to Generate a Populate Script

A database is more useful with data in it. Use **AI** to draft a script that **populates** (seeds) your cities and teams.

- Give it your **schema** and a clear ask: *"write a Prisma script that inserts 5 cities and 15 teams across different sports"*
- It can generate realistic sample data quickly
- **Review, run, and verify** with a query before trusting it
- Disclose your AI use, and **trust but verify**

SQL Games

Keep practicing SQL by playing:

- **SQL Police Department:** <https://sqlpd.com/>
- **SQL Murder Mystery:** <https://mystery.knightlab.com/>